

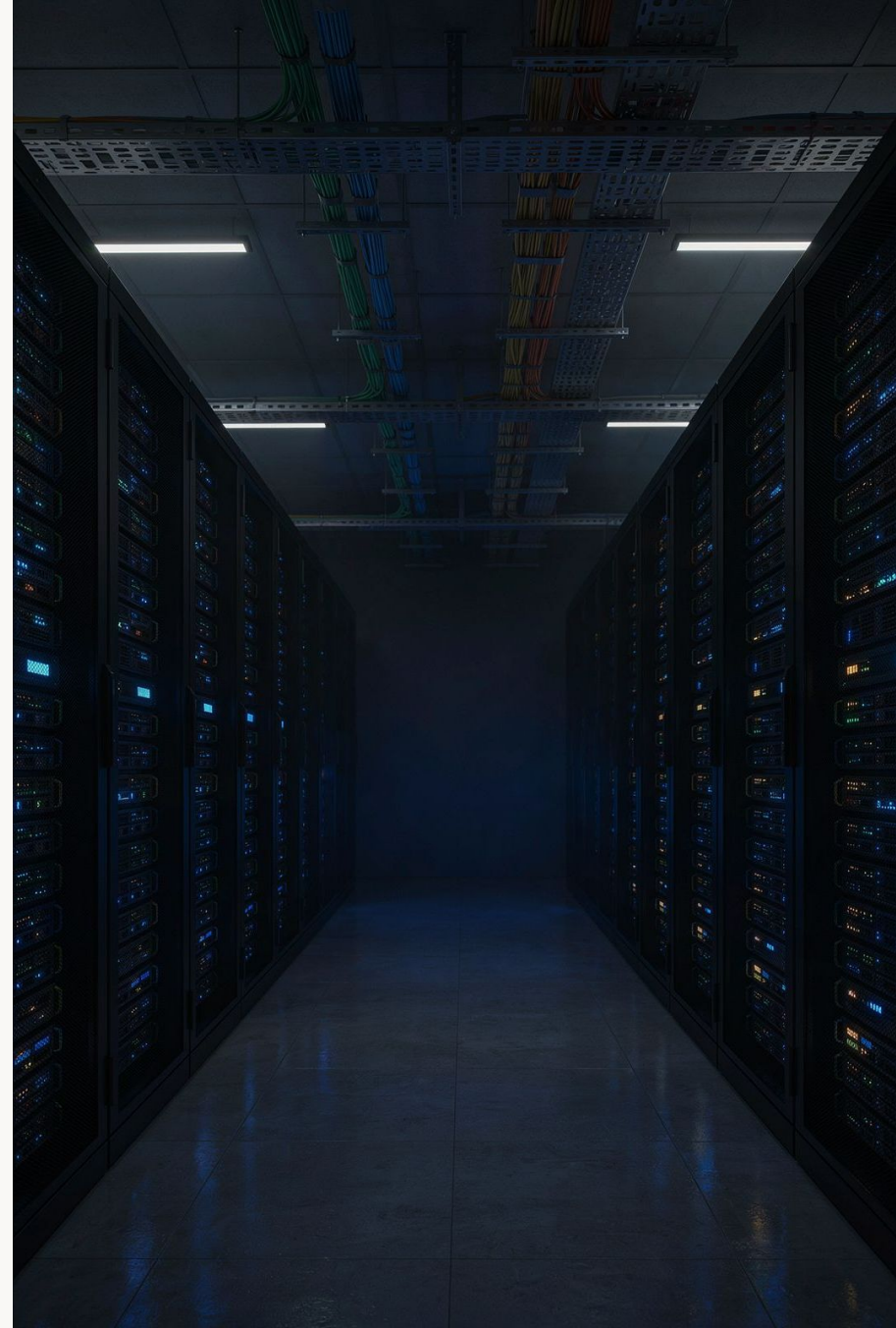
Trigger, View e Stored Procedure

Automatizando tarefas e organizando consultas no MySQL

BANCO DE DADOS

MYSQL

AVANÇADO



Situação-Problema

Uma loja de informática possui um sistema com clientes, produtos, pedidos e itens vendidos. O banco de dados precisa resolver desafios reais do dia a dia:

Relatórios sem repetição

Mostrar relatórios de vendas sem repetir consultas enormes a cada acesso.

Histórico de pedidos

Consultar rapidamente todos os pedidos de cada cliente.

Controle de estoque

Impedir vendas quando não houver estoque disponível e diminuir automaticamente após a venda.

Auditoria

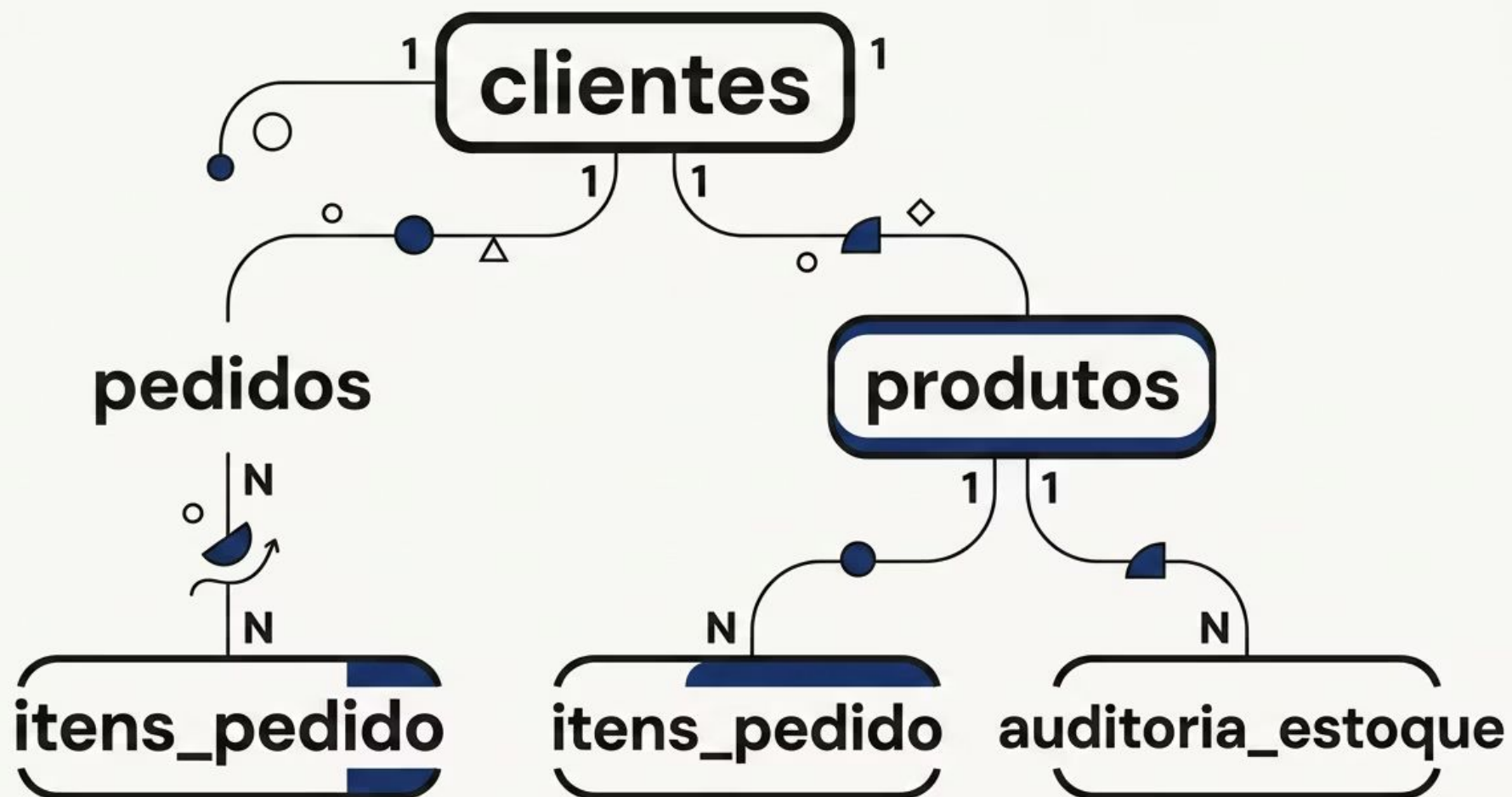
Registrar automaticamente todas as alterações feitas no estoque.

 Como automatizar essas tarefas diretamente no banco de dados? A resposta está em **Trigger, View e Stored Procedure**.



Banco de Dados LojaTech

Todas as aulas utilizam um banco de dados chamado **loja_tech**, composto por cinco tabelas interligadas:



Os relacionamentos seguem o padrão: um cliente realiza vários pedidos; cada pedido possui vários itens; cada item aponta para um produto; e toda alteração de estoque é registrada em auditoria.

CONFIGURAÇÃO INICIAL

Criação do Banco de Dados

Código MySQL

```
CREATE DATABASE loja_tech;  
  
USE loja_tech;
```

Execute esses dois comandos antes de qualquer outro. Eles preparam o ambiente de trabalho.

O que cada comando faz?

1 CREATE DATABASE

Cria o banco de dados chamado **loja_tech** no servidor MySQL.

2 USE

Seleciona o banco recém-criado. Todos os próximos comandos serão executados dentro dele.

Criação das Tabelas

As cinco tabelas abaixo formam a base do sistema LojaTech. Observe as chaves primárias (**PRIMARY KEY**), estrangeiras (**FOREIGN KEY**) e restrições de integridade:

```
CREATE TABLE clientes (  
  id_cliente INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(100) UNIQUE,  
  cidade VARCHAR(80)  
);  
  
CREATE TABLE produtos (  
  id_produto INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  categoria VARCHAR(50),  
  preco DECIMAL(10,2) NOT NULL,  
  estoque INT NOT NULL DEFAULT 0  
);  
  
CREATE TABLE pedidos (  
  id_pedido INT AUTO_INCREMENT PRIMARY KEY,  
  id_cliente INT NOT NULL,  
  data_pedido DATETIME DEFAULT CURRENT_TIMESTAMP,  
  status VARCHAR(30) DEFAULT 'ABERTO',  
  CONSTRAINT fk_pedido_cliente  
  FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente)  
);  
  
CREATE TABLE itens_pedido (  
  id_item INT AUTO_INCREMENT PRIMARY KEY,  
  id_pedido INT NOT NULL,  
  id_produto INT NOT NULL,  
  quantidade INT NOT NULL,  
  preco_unitario DECIMAL(10,2) NOT NULL,  
  CONSTRAINT fk_item_pedido FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido),  
  CONSTRAINT fk_item_produto FOREIGN KEY (id_produto) REFERENCES produtos(id_produto)  
);  
  
CREATE TABLE auditoria_estoque (  
  id_auditoria INT AUTO_INCREMENT PRIMARY KEY,  
  id_produto INT NOT NULL,  
  estoque_anterior INT,  
  estoque_novo INT,  
  data_alteracao DATETIME DEFAULT CURRENT_TIMESTAMP,  
  usuario VARCHAR(100),  
  CONSTRAINT fk_auditoria_produto  
  FOREIGN KEY (id_produto) REFERENCES produtos(id_produto)  
);
```

Inserindo Dados de Exemplo

```
INSERT INTO clientes (nome, email, cidade) VALUES  
( 'Ana Souza', 'ana@email.com', 'São Paulo'),  
( 'Bruno Lima', 'bruno@email.com', 'São José dos Campos'),  
( 'Carla Mendes', 'carla@email.com', 'Taubaté');
```

```
INSERT INTO produtos (nome, categoria, preco, estoque) VALUES  
( 'Notebook Gamer', 'Informática', 4500.00, 10),  
( 'Mouse Sem Fio', 'Periféricos', 89.90, 30),  
( 'Teclado Mecânico', 'Periféricos', 249.90, 20),  
( 'Monitor 24 Polegadas', 'Informática', 899.90, 15);
```

```
INSERT INTO pedidos (id_cliente, status) VALUES  
(1, 'FINALIZADO'), (2, 'ABERTO');
```

```
INSERT INTO itens_pedido (id_pedido, id_produto, quantidade, preco_unitario) VALUES  
(1, 1, 1, 4500.00),  
(1, 2, 2, 89.90),  
(2, 3, 1, 249.90);
```

 Esses dados serão utilizados em todos os exemplos de **View**, **Stored Procedure** e **Trigger** ao longo da aula.



PARTE 1

O que é uma View?

Uma janela para seus dados

Conceito de View

Definição

Uma **View** é uma tabela virtual criada a partir de uma consulta SQL. Ela não armazena dados próprios — apresenta dados organizados que estão em outras tabelas.

Analogia do cotidiano

Pense em uma janela: ela não cria uma nova paisagem, apenas mostra uma parte organizada da paisagem que já existe.

Quando usar?

- **Simplificar consultas complexas**
Evite repetir JOINS longos toda vez.
- **Ocultar colunas sensíveis**
Exponha apenas o necessário para cada usuário.
- **Criar relatórios padronizados**
Uma View vira o padrão oficial da empresa.
- **Facilitar o acesso**
Usuários sem domínio de SQL consultam dados com facilidade.

Criando uma View

A View abaixo reúne dados de quatro tabelas em uma única consulta organizada, calculando o subtotal de cada item vendido:

```
CREATE VIEW vw_detalhes_vendas AS
SELECT
  p.id_pedido,
  p.data_pedido,
  c.nome      AS cliente,
  pr.nome     AS produto,
  ip.quantidade,
  ip.preco_unitario,
  ip.quantidade * ip.preco_unitario AS subtotal,
  p.status
FROM pedidos p
INNER JOIN clientes c  ON p.id_cliente = c.id_cliente
INNER JOIN itens_pedido ip ON p.id_pedido = ip.id_pedido
INNER JOIN produtos pr  ON ip.id_produto = pr.id_produto;
```

4 tabelas

Reunidas em uma View

subtotal

Campo calculado automaticamente

Nomes claros

Aliases facilitam relatórios

Consultando uma View

Consultas possíveis

```
-- Todos os dados  
SELECT * FROM vw_detalhes_vendas;
```

```
-- Apenas pedidos finalizados  
SELECT *  
FROM vw_detalhes_vendas  
WHERE status = 'FINALIZADO';
```

```
-- Total comprado por cliente  
SELECT cliente,  
       SUM(subtotal) AS total_comprado  
FROM vw_detalhes_vendas  
GROUP BY cliente;
```

Resultado simulado

Pedido	Cliente	Produto	Qtd	Subtotal
1	Ana Souza	Notebook Gamer	1	R\$ 4.500,00
1	Ana Souza	Mouse Sem Fio	2	R\$ 179,80
2	Bruno Lima	Teclado Mecânico	1	R\$ 249,90

Perceba que a View é consultada exatamente como uma tabela comum, usando **SELECT**, **WHERE** e **GROUP BY**.

Alterando e Excluindo uma View

Criar ou substituir

```
CREATE OR REPLACE VIEW vw_produtos_disponiveis AS
SELECT
  id_produto,
  nome,
  categoria,
  preco,
  estoque
FROM produtos
WHERE estoque > 0;

-- Consultando
SELECT * FROM vw_produtos_disponiveis;
```

Excluindo a View

```
DROP VIEW vw_produtos_disponiveis;
```

✓ Excluir uma View **não exclui** as tabelas nem os dados originais. Apenas a definição da consulta é removida.

Resumo dos comandos

CREATE VIEW

Cria uma nova View

CREATE OR REPLACE

Atualiza uma View existente

DROP VIEW

Remove a View do banco



PARTE 2

O que é uma Stored Procedure?

Uma receita pronta no banco

Conceito de Stored Procedure

Definição

Uma **Stored Procedure** (procedimento armazenado) é um conjunto de comandos SQL salvo diretamente no banco de dados, pronto para ser reutilizado.

Analogia do cotidiano

Como uma receita pronta: você informa os ingredientes (parâmetros), executa a receita e recebe o resultado — sem precisar reescrever tudo.

O que uma Procedure pode fazer?

01

Receber parâmetros

Entrada (IN), saída (OUT) ou ambos (INOUT).

02

Executar múltiplos comandos

SELECT, INSERT, UPDATE e DELETE em sequência.

03

Usar condições e cálculos

IF, ELSE, loops e operações matemáticas.

04

Centralizar regras de negócio

A lógica fica no banco, acessível a qualquer aplicação.

Procedure com Parâmetro de Entrada (IN)

Esta procedure recebe o código de um cliente e retorna todos os pedidos dele com o total de cada um:

```
DELIMITER //
CREATE PROCEDURE sp_pedidos_cliente(
  IN p_id_cliente INT
)
BEGIN
  SELECT
    p.id_pedido,
    p.data_pedido,
    p.status,
    SUM(ip.quantidade * ip.preco_unitario) AS total_pedido
  FROM pedidos p
  INNER JOIN itens_pedido ip ON p.id_pedido = ip.id_pedido
  WHERE p.id_cliente = p_id_cliente
  GROUP BY p.id_pedido, p.data_pedido, p.status;
END //
DELIMITER ;
```

Executando

```
CALL sp_pedidos_cliente(1);
```

Pontos importantes

- **IN**: parâmetro de entrada — recebe o código do cliente
- **BEGIN / END**: delimitam o bloco de comandos
- **DELIMITER**: permite usar ; dentro da procedure sem encerrá-la antes do tempo

Procedure para Reajuste de Preços

Procedure com dois parâmetros de entrada, validação condicional e tratamento de erro com **SIGNAL**:

```
DELIMITER //
CREATE PROCEDURE sp_reajustar_precos(
  IN p_categoria VARCHAR(50),
  IN p_percentual DECIMAL(5,2)
)
BEGIN
  IF p_percentual <= 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'O percentual deve ser maior que zero';
  ELSE
    UPDATE produtos
      SET preco = preco + (preco * p_percentual / 100)
      WHERE categoria = p_categoria;
  END IF;
END //
DELIMITER ;
```

Executando um reajuste de 10%

```
CALL sp_reajustar_precos('Periféricos', 10);

-- Verificando o resultado
SELECT nome, categoria, preco
FROM produtos
WHERE categoria = 'Periféricos';
```

Estruturas utilizadas

IF / ELSE

Executa lógica condicional dentro da procedure

SIGNAL

Lança um erro personalizado quando a regra é violada

Procedure com Parâmetro de Saída (OUT)

Código da procedure

```
DELIMITER //
CREATE PROCEDURE sp_total_clientes(
  OUT p_quantidade INT
)
BEGIN
  SELECT COUNT(*)
  INTO p_quantidade
  FROM clientes;
END //
DELIMITER ;
```

Executando e consultando

```
CALL sp_total_clientes(@total);

SELECT @total AS quantidade_clientes;
```

Como funciona o OUT?

1 Parâmetro OUT

Diferente de **IN**, o **OUT** devolve um valor para quem chamou a procedure — ideal para retornar resultados calculados.

2 Cláusula INTO

Armazena o resultado do SELECT diretamente na variável de saída `p_quantidade`.

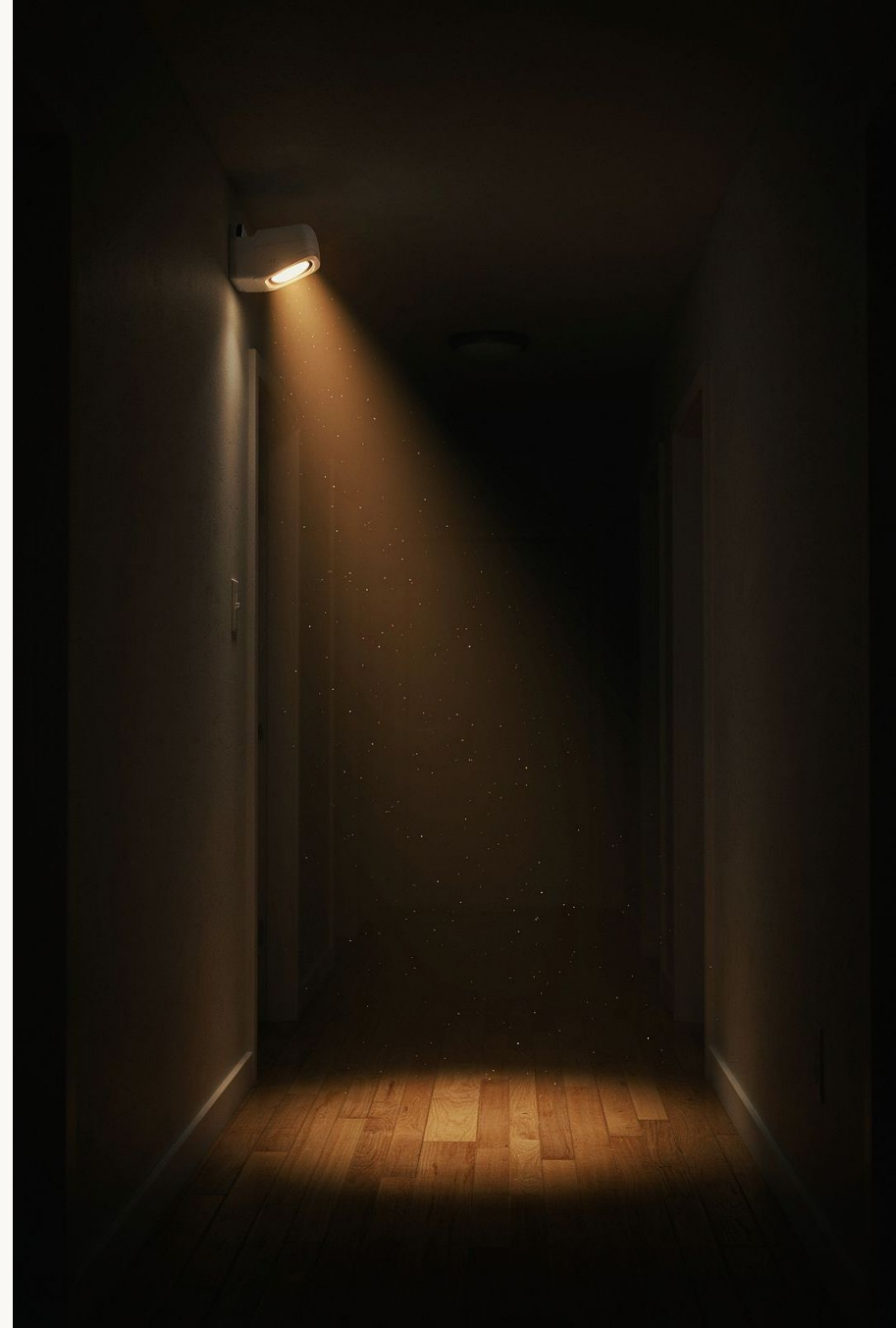
3 Variável de sessão @total

No MySQL, variáveis iniciadas com `@` são variáveis de sessão — persistem durante toda a conexão.

PARTE 3

O que é uma Trigger?

Um sensor automático
no banco



Conceito de Trigger

Definição

Uma **Trigger** (gatilho) é um comando executado **automaticamente** quando um evento ocorre em uma tabela: INSERT, UPDATE ou DELETE.

Analogia do cotidiano

Como um sensor de presença: quando alguém entra no ambiente, uma ação é disparada automaticamente — sem nenhuma intervenção manual.

Eventos e momentos

Momento	Descrição
BEFORE	Executa <i>antes</i> do evento — ideal para validações
AFTER	Executa <i>depois</i> do evento — ideal para reações

Exemplos de uso

- Atualizar estoque automaticamente
- Criar registros de auditoria
- Impedir dados inválidos
- Calcular valores automaticamente

Estrutura Básica de uma Trigger

Sintaxe

```
DELIMITER //  
CREATE TRIGGER nome_da_trigger  
BEFORE INSERT ON nome_da_tabela  
FOR EACH ROW  
BEGIN  
    -- comandos executados automaticamente  
END //  
DELIMITER ;
```

 **FOR EACH ROW:** a Trigger executa para cada linha afetada pelo evento — não apenas uma vez por comando.

Disponibilidade de OLD e NEW

Evento	OLD	NEW
INSERT	✗ Não disponível	✓ Disponível
UPDATE	✓ Disponível	✓ Disponível
DELETE	✓ Disponível	✗ Não disponível

OLD acessa os valores anteriores; **NEW** acessa os novos valores que serão inseridos ou atualizados.

Trigger: Impedir Venda sem Estoque

Esta Trigger é executada **antes** de inserir um item no pedido. Ela valida a quantidade e verifica se o estoque é suficiente:

```
DELIMITER //
CREATE TRIGGER trg_verificar_estoque
BEFORE INSERT ON itens_pedido
FOR EACH ROW
BEGIN
    DECLARE v_estoque_atual INT;

    SELECT estoque INTO v_estoque_atual
    FROM produtos
    WHERE id_produto = NEW.id_produto;

    IF NEW.quantidade <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'A quantidade deve ser maior que zero';
    END IF;

    IF v_estoque_atual < NEW.quantidade THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Estoque insuficiente para realizar a venda';
    END IF;
END //
DELIMITER ;
```

01

Disparada BEFORE INSERT

Executa antes do item ser salvo na tabela.

03

Valida a quantidade

Rejeita valores menores ou iguais a zero.

02

Consulta o estoque

Busca o estoque atual usando `NEW.id_produto`.

04

Verifica disponibilidade

Impede a operação se o estoque for insuficiente.

Trigger: Diminuir o Estoque Automaticamente

Código da Trigger

```
DELIMITER //  
CREATE TRIGGER trg_diminuir_estoque  
AFTER INSERT ON itens_pedido  
FOR EACH ROW  
BEGIN  
    UPDATE produtos  
    SET estoque = estoque - NEW.quantidade  
    WHERE id_produto = NEW.id_produto;  
END //  
DELIMITER ;
```

Testando na prática

```
-- Novo pedido para Carla (id=3)  
INSERT INTO pedidos (id_cliente, status)  
VALUES (3, 'ABERTO');  
  
-- Inserindo 3 unidades do Mouse Sem Fio  
INSERT INTO itens_pedido  
    (id_pedido, id_produto, quantidade, preco_unitario)  
VALUES (3, 2, 3, 89.90);  
  
-- Verificando o estoque atualizado  
SELECT id_produto, nome, estoque  
FROM produtos  
WHERE id_produto = 2;
```

✅ O estoque foi de **30** para **27** automaticamente — sem nenhum UPDATE manual!

Trigger de Auditoria de Estoque

Toda vez que o estoque de um produto for alterado, esta Trigger registra automaticamente quem fez a mudança e quais foram os valores:

```
DELIMITER //
CREATE TRIGGER trg_auditar_estoque
AFTER UPDATE ON produtos
FOR EACH ROW
BEGIN
    IF OLD.estoque <> NEW.estoque THEN
        INSERT INTO auditoria_estoque
            (id_produto, estoque_anterior, estoque_novo, usuario)
        VALUES
            (NEW.id_produto, OLD.estoque, NEW.estoque, USER());
    END IF;
END //
DELIMITER ;
```

Consultando o histórico

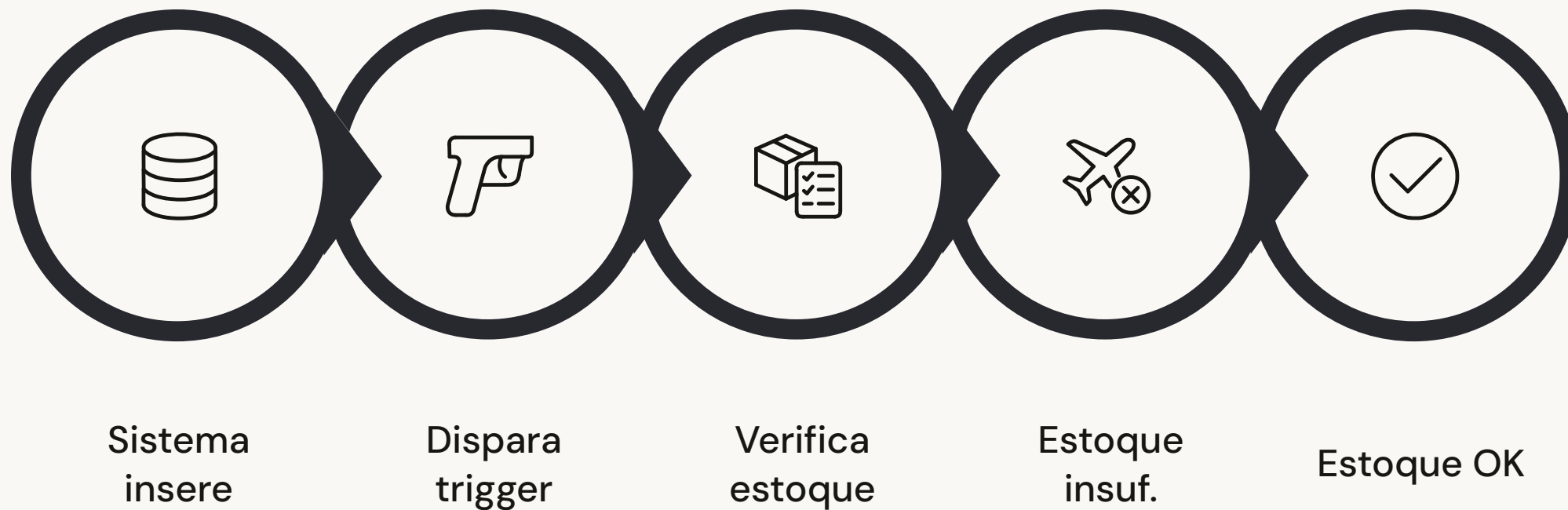
```
SELECT a.id_auditoria,
       p.nome AS produto,
       a.estoque_anterior,
       a.estoque_novo,
       a.data_alteracao,
       a.usuario
FROM auditoria_estoque a
INNER JOIN produtos p ON a.id_produto = p.id_produto
ORDER BY a.data_alteracao DESC;
```

Como funciona?

- **OLD.estoque:** valor antes da alteração
- **NEW.estoque:** valor após a alteração
- **USER():** identifica o usuário MySQL conectado
- O IF garante que o registro só ocorre quando o estoque realmente muda

Fluxo Completo de uma Venda

Veja como uma única operação de inserção ativa uma cadeia de automações em sequência:



📌 Uma única operação pode ativar **diferentes automações em sequência**. Planeje e documente bem para evitar efeitos inesperados.

Consultando e Excluindo Triggers e Procedures

Gerenciando Triggers

```
-- Listar todas as Triggers do banco  
SHOW TRIGGERS;  
  
-- Excluir uma Trigger específica  
DROP TRIGGER trg_diminuir_estoque;
```

Gerenciando Stored Procedures

```
-- Listar Procedures do banco loja_tech  
SHOW PROCEDURE STATUS  
WHERE Db = 'loja_tech';  
  
-- Excluir uma Procedure específica  
DROP PROCEDURE sp_pedidos_cliente;
```

Gerenciando Views

```
-- Listar todas as Views  
SHOW FULL TABLES  
WHERE Table_type = 'VIEW';  
  
-- Ver a definição de uma View  
SHOW CREATE VIEW vw_detalhes_vendas;  
  
-- Excluir uma View  
DROP VIEW vw_detalhes_vendas;
```



Tenha cuidado ao excluir objetos. A aplicação pode depender de Triggers, Procedures ou Views que você está removendo. Sempre verifique antes de excluir.

Comparação Geral

Recurso	Principal Função	Execução	Exemplo Prático
View	Simplificar e organizar consultas	<code>SELECT * FROM view</code>	Relatório de vendas por cliente
Stored Procedure	Agrupar comandos e regras de negócio	<code>CALL procedure()</code>	Consultar pedidos de um cliente
Trigger	Automatizar ações após eventos	Executada automaticamente	Atualizar estoque após venda



View

Uma *janela* organizada para seus dados



Stored Procedure

Uma *receita pronta* que você executa com parâmetros



Trigger

Um *sensor automático* que reage a eventos

Quando Utilizar Cada Recurso?



Use View quando...

- Uma consulta é muito grande ou complexa
- O mesmo relatório é acessado várias vezes
- Deseja ocultar colunas sensíveis
- Precisa padronizar o acesso aos dados



Use Stored Procedure quando...

- Vários comandos precisam ser executados juntos
- Existe uma regra de negócio reutilizável
- É necessário receber e retornar parâmetros
- Deseja centralizar operações no banco



Use Trigger quando...

- Uma ação deve acontecer automaticamente
- É necessário registrar auditoria de alterações
- Deseja impedir dados inválidos na inserção
- Uma tabela precisa reagir à mudança em outra

Vantagens e Cuidados

✓ Vantagens

- **Menos repetição de código**
Escreva uma vez, reutilize sempre.
- **Automação de tarefas**
O banco trabalha sozinho em resposta a eventos.
- **Centralização de regras**
Regras de negócio ficam no banco, não dispersas no código.
- **Segurança e padronização**
Operações passam por validações automáticas.

⚠ Cuidados

- **Triggers invisíveis**
Podem executar ações que a aplicação não vê diretamente.
- **Excesso dificulta manutenção**
Muitas Triggers encadeadas tornam o sistema difícil de depurar.
- **Views complexas**
Podem apresentar problemas de desempenho em grandes volumes.
- **Documente tudo**
Sempre registre o que cada objeto faz e por quê.

Automação sem documentação pode transformar facilidade em dificuldade.

Erros Comuns — Fique Atento!

Esquecer o DELIMITER

Sem alterar o delimitador, o MySQL interpreta o primeiro ; como fim da procedure antes do esperado.

```
DELIMITER //  
-- ... código ...  
DELIMITER ;
```

Usar OLD ou NEW no evento errado

INSERT → só tem **NEW**. **DELETE** → só tem **OLD**. **UPDATE** → tem os dois. Usar o errado causa erro imediato.

Trigger duplicada

O MySQL não permite duas Triggers com o mesmo nome no mesmo banco. Use **DROP TRIGGER** antes de recriar.

Não validar valores nulos ou negativos

Sempre use **IF** e **SIGNAL** para proteger campos críticos como preço e quantidade.

Regras escondidas sem documentação

Documente todas as Triggers e Procedures com comentários e um registro centralizado das automações ativas.

DESAFIO 1

Desafio Prático — View de Estoque Baixo

A empresa precisa monitorar produtos com pouco estoque. Crie uma View chamada **vw_estoque_baixo** que exiba apenas produtos com estoque menor ou igual a 10, com as colunas: código, nome, categoria, estoque e preço.

❓ Tente criar a View antes de ver a resposta! Lembre-se: use `WHERE estoque <= 10`.

Resposta

```
CREATE VIEW vw_estoque_baixo AS
SELECT
  id_produto,
  nome,
  categoria,
  estoque,
  preco
FROM produtos
WHERE estoque <= 10;

-- Consultando
SELECT * FROM vw_estoque_baixo;
```

Desafio Prático — Procedure por Categoria

Crie uma Stored Procedure chamada **sp_produtos_categoria** que receba uma categoria como parâmetro e retorne os produtos daquela categoria, ordenados do menor para o maior preço.

? Tente escrever a procedure completa antes de conferir a resposta!

Resposta

```
DELIMITER //
CREATE PROCEDURE sp_produtos_categoria(
  IN p_categoria VARCHAR(50)
)
BEGIN
  SELECT
    id_produto,
    nome,
    preco,
    estoque
  FROM produtos
  WHERE categoria = p_categoria
  ORDER BY preco ASC;
END //
DELIMITER ;

-- Executando
CALL sp_produtos_categoria('Periféricos');
```


Desafio Prático — Trigger de Validação de Preço

Crie uma Trigger que impeça o cadastro de produtos com preço menor ou igual a zero. O banco deve rejeitar a operação e exibir uma mensagem de erro clara.

? Lembre-se: use `BEFORE INSERT` e `SIGNAL SQLSTATE '45000'!`

Resposta

```
DELIMITER //
CREATE TRIGGER trg_validar_preco_produto
BEFORE INSERT ON produtos
FOR EACH ROW
BEGIN
  IF NEW.preco <= 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'O preço do produto deve ser maior que zero';
  END IF;
END //
DELIMITER ;

-- Teste inválido (deve ser rejeitado pelo banco)
INSERT INTO produtos (nome, categoria, preco, estoque)
VALUES ('Produto de Teste', 'Teste', -10.00, 5);
```

Quiz — Teste seus Conhecimentos!

Qual recurso representa uma tabela virtual?

- 1 A) Trigger **B) View** ☒ C) Stored Procedure D) Primary Key

Uma View é uma tabela virtual baseada em uma consulta SQL. Ela não armazena dados próprios.

Qual comando executa uma Stored Procedure?

- 2 A) EXECUTE TABLE B) RUN PROCEDURE **C) CALL** ☒ D) START

O comando correto é `CALL nome_da_procedure()`. Os demais não existem no MySQL padrão.

Qual palavra representa o valor anterior em uma Trigger de UPDATE?

- 3 A) BEFORE B) NEW **C) OLD** ☒ D) PREVIOUS

OLD representa o valor antes da alteração; NEW representa o valor após.

Qual recurso é executado automaticamente após um evento?

- 4 A) View **B) Trigger** ☒ C) SELECT D) Stored Procedure

A Trigger é disparada automaticamente pelo banco de dados em resposta a INSERT, UPDATE ou DELETE.

Qual é a principal vantagem de uma View?

- 5 A) Criar um novo servidor B) Substituir todas as tabelas **C) Simplificar consultas complexas** ☒ D) Excluir dados automaticamente

A View organiza e simplifica o acesso aos dados, evitando repetição de JOINS e lógicas complexas.

Resumo Final

View

Cria uma visão organizada dos dados existentes.

```
SELECT * FROM nome_da_view;
```

- Tabela virtual
- Não armazena dados
- Ideal para relatórios

Stored Procedure

Armazena e executa um conjunto de comandos com parâmetros.

```
CALL nome_da_procedure();
```

- Recebe parâmetros IN/OUT
- Centraliza regras de negócio
- Executa múltiplos comandos

Trigger

Executa ações automaticamente após eventos na tabela.

```
BEFORE/AFTER INSERT  
BEFORE/AFTER UPDATE  
BEFORE/AFTER DELETE
```

- Automação total
- Usa OLD e NEW
- Ideal para auditoria

View organiza, Stored Procedure executa e Trigger automatiza.



Agora é sua vez!

Escolha um sistema — uma biblioteca, uma clínica, um e-commerce — e aplique os três recursos aprendidos hoje: crie uma **View** para organizar os dados, uma **Stored Procedure** para automatizar uma consulta e uma **Trigger** para proteger a integridade do sistema.

Crie uma View

Organize seus dados em uma consulta reutilizável



Crie uma Procedure

Centralize uma regra de negócio do seu sistema

Crie uma Trigger

Automatize uma ação ou registre uma auditoria

BONS ESTUDOS! 🚀